République Algérienne Démocratique et Populaire

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université Ben Youcef Benkhedda – Alger 1



Faculté des Sciences

Département d'Informatique

ISII M2

Rapport du TP1 MBI

Module : Méthodes Bio-Inspirées

2ème Année Master en Informatique

Année Universitaire : 2026/2027

Réalisé par :

SELMOUNE Meriem (G1)
KELLOU Abdelghani (G3)
LAMMALI Melissa (G3)
SI MOHAMED Nesrine (G1)
BOUNEGAB Sarra (G2)

Date de remise : 2 novembre 2026

Contents

1 Introduction **3**

2 Outils et environnement **3**

I Bases de la théorie des graphes **3**

3 Rappels théoriques **3**

4 Création d'un graphe non orienté G et d'un graphe orienté DG **3**

5 Suppression du sommet 1 du graphe G **4**

6 Ajout des arêtes au graphe G **4**

7 Ajout des arcs au graphe DG **4**

8 Tracé des graphes G et DG **5**

9 Taille du graphe G **5**

10 Ordre des graphes G et DG **5**

11 Matrice d'adjacence du graphe G **6**

12 Pondération des arcs du graphe DG **6**

13 Matrice d'adjacence du graphe DG **7**

14 Listes d'adjacence du graphe DG **7**

II Parcours en profondeur **7**

15 Question 1 : Implémentation de G en utilisant un dictionnaire **8**

16 Question 2 : Fonction `ParcourirProfondeur(Graphe, Sommet, Visitee)` **9**
16.1 Code de la fonction 9
16.2 Fonctionnement 10

17 Question 3 : Application de la fonction sur le graphe G **10**
17.1 Code utilisé 10
17.2 Résultat de l'exécution 10
17.3 Interprétation 11

Conclusion **12**

List of Figures

1 Création des graphes G et DG et ajout des sommets 4
2 Sommets de G et de DG après la suppression du sommet 1 de G 4
3 Ajout des arêtes au graphe G 4
4 Ajout des arcs au graphe orienté DG 4
5 Tracé du graphe non orienté G et du graphe orienté DG 5
6 Calcul de la taille du graphe G 5
7 Calcul de l'ordre des graphes G et DG 6
8 Matrice d'adjacence du graphe G 6
9 Pondération des arcs du graphe DG 6
10 Matrice d'adjacence du graphe orienté DG 7
11 Listes d'adjacence du graphe orienté DG 7
12 Affichage du dictionnaire représentant le graphe G 8
13 Tracé du graphe G construit à partir du dictionnaire 9
14 Code utilisé pour lancer le parcours en profondeur depuis a puis depuis f 10
15 Résultat de l'exécution du parcours en profondeur depuis a puis depuis f 10

1 Introduction

Les méthodes bio-inspirées, comme les algorithmes génétiques ou l'optimisation par colonies de fourmis, résolvent des problèmes complexes en imitant des mécanismes naturels. Beaucoup d'entre elles opèrent sur des graphes : l'optimisation par colonies de fourmis, par exemple, cherche des chemins sur un graphe. Maîtriser leur manipulation est donc un préalable à ces algorithmes.

L'objectif de ce TP n°1 est de se familiariser avec les graphes sous Python à l'aide de la bibliothèque **NetworkX**. La première partie porte sur les bases de la théorie des graphes : création de graphes orientés et non orientés, gestion des sommets et des arêtes, calcul de l'ordre et de la taille, matrices et listes d'adjacence. La seconde partie implémente le parcours en profondeur d'un graphe représenté par un dictionnaire.

2 Outils et environnement

Ce TP est réalisé en Python dans un notebook Jupyter (VS Code). La bibliothèque **NetworkX** permet de créer, manipuler et étudier des graphes. Deux de ses classes sont utilisées : `Graph` pour les graphes non orientés et `DiGraph` pour les graphes orientés. Elle fournit les opérations nécessaires à la partie 1 : ajout et suppression de sommets et d'arêtes, calcul de l'ordre et de la taille, matrices et listes d'adjacence. Le tracé des graphes s'appuie sur **Matplotlib**.

La partie 2 s'appuie sur NetworkX uniquement pour le tracé : conformément à l'énoncé, le graphe est représenté par un dictionnaire Python et le parcours en profondeur est implémenté par une fonction écrite manuellement, sans recourir aux algorithmes de parcours de la bibliothèque. NetworkX et Matplotlib servent seulement à vérifier visuellement que le dictionnaire correspond au graphe de l'énoncé.

Versions utilisées : Python `3.13.5`, NetworkX `3.4.2`.

Part I
Bases de la théorie des graphes

3 Rappels théoriques

Un graphe est défini par un couple $G = (V, E)$, où V est l'ensemble des sommets (individus, villes, produits, etc.) et E l'ensemble des arêtes, chacune reliant une paire de sommets. Le cardinal $|V|$ est l'**ordre** du graphe et $|E|$ sa **taille**.

Un graphe est **non orienté** lorsque ses arêtes sont symétriques : l'arête $\{u, v\}$ relie u et v dans les deux sens. Il est **orienté** lorsque ses liens, appelés **arcs**, ont un sens : l'arc (u, v) va de u vers v et diffère de (v, u).

Deux représentations sont utilisées dans ce TP. La **matrice d'adjacence** est une matrice carrée A d'ordre $|V|$ où A_{ij} vaut 1 (ou le poids du lien, si le graphe est pondéré) lorsque i est relié à j, et 0 sinon ; elle est symétrique pour un graphe non orienté, mais pas nécessairement pour un graphe orienté. Les **listes d'adjacence** associent à chaque sommet la liste de ses voisins (ses successeurs, dans le cas orienté).

4 Création d'un graphe non orienté G et d'un graphe orienté DG

On utilise la classe `Graph()` pour créer le graphe non orienté et `DiGraph()` pour le graphe orienté. On ajoute ensuite les sommets 1, 2, 3, 4 et 5 à chacun avec la méthode `add_nodes_from()`, puis on les affiche avec `G.nodes` et `DG.nodes`. Le résultat est donné en figure 1.



Figure 1: Création des graphes G et DG et ajout des sommets

On constate que les deux graphes contiennent bien les sommets 1 à 5, soit un ordre de 5 chacun.

5 Suppression du sommet 1 du graphe G

On utilise la méthode `remove_node()` en lui passant la valeur du sommet à supprimer. Cette suppression ne concerne que G : le graphe DG n'est pas modifié (figure 2).

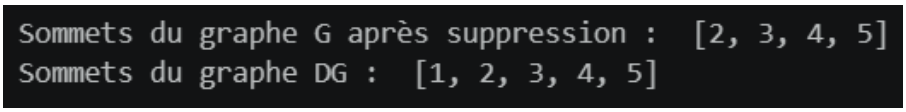


Figure 2: Sommets de G et de DG après la suppression du sommet 1 de G

On constate que G ne contient plus que les sommets 2, 3, 4 et 5, alors que DG conserve les cinq sommets, dont le sommet 1.

6 Ajout des arêtes au graphe G

On utilise la méthode `add_edges_from()` avec la liste des arêtes $\{2,3\}$, $\{2,5\}$, $\{3,4\}$ et $\{4,5\}$, puis on les affiche avec `G.edges`. Comme G est non orienté, `(2, 3)` et `(3, 2)` désignent la même arête. Le résultat est donné en figure 3.



Figure 3: Ajout des arêtes au graphe G

7 Ajout des arcs au graphe DG

On utilise la même méthode `add_edges_from()` sur le graphe orienté avec les arcs $(1,3)$, $(2,3)$, $(2,4)$, $(2,5)$, $(4,5)$ et $(5,1)$. Contrairement à G, l'ordre des sommets dans chaque couple est significatif : $(5,1)$ va de 5 vers 1 et n'est pas équivalent à $(1,5)$. Le résultat est donné en figure 4.

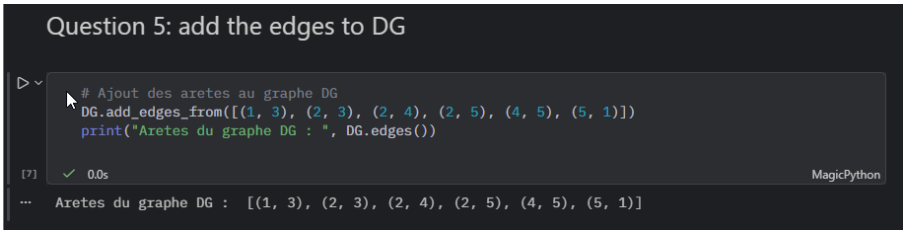


Figure 4: Ajout des arcs au graphe orienté DG

On constate que DG contient bien les six arcs demandés, dont l'arc $(5,1)$ qui va du sommet 5 vers le sommet 1, et non l'inverse.

8 Tracé des graphes G et DG

Le tracé s'appuie sur `nx.draw()` de NetworkX, associé à Matplotlib : l'option `with_labels=True` affiche le nom des sommets, et les arcs de DG sont représentés par des flèches. Les deux graphes sont tracés côte à côte à l'aide de `plt.subplot()` (figure 5).

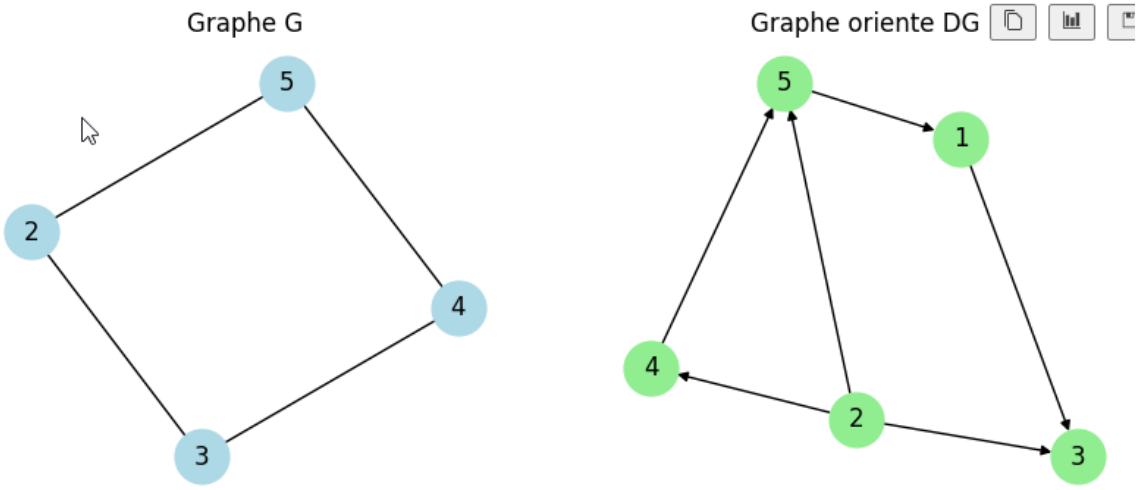


Figure 5: Tracé du graphe non orienté G et du graphe orienté DG

Le graphe G ne comporte que les sommets 2, 3, 4 et 5 reliés par des traits simples, alors que DG comporte les cinq sommets reliés par des flèches indiquant le sens des arcs.

9 Taille du graphe G

La taille d'un graphe correspond à son nombre d'arêtes. Elle est obtenue avec la méthode `G.size()` (figure 6).

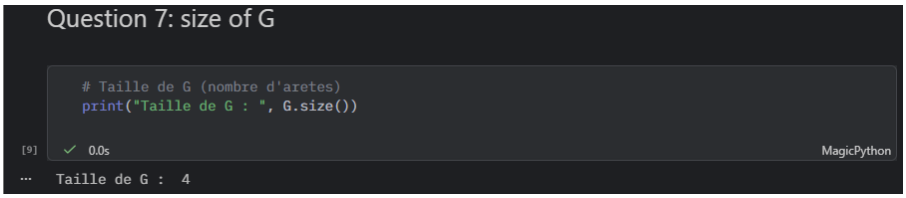


Figure 6: Calcul de la taille du graphe G

On obtient une taille de 4, ce qui correspond aux quatre arêtes $\{2,3\}$, $\{2,5\}$, $\{3,4\}$ et $\{4,5\}$ ajoutées précédemment.

10 Ordre des graphes G et DG

L'ordre d'un graphe correspond à son nombre de sommets. Il est obtenu avec la méthode `order()`, appelée sur G et sur DG (figure 7).

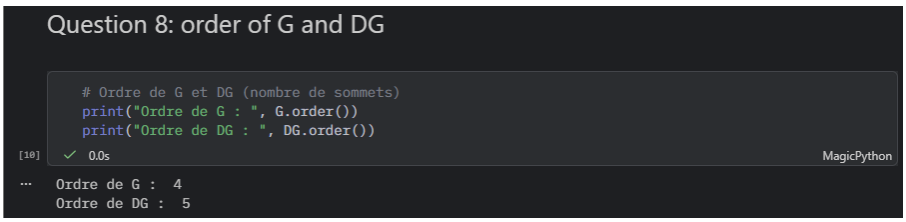


Figure 7: Calcul de l'ordre des graphes G et DG

On obtient un ordre de 4 pour G et de 5 pour DG. Cet écart s'explique par la suppression du sommet 1 de G, qui n'a pas été effectuée sur DG.

11 Matrice d'adjacence du graphe G

Nous utilisons la fonction `adjacency_matrix()` de NetworkX en lui passant le graphe G en paramètre. Elle renvoie une matrice creuse SciPy, dont l'affichage direct est une liste de triplets (ligne, colonne, valeur) peu lisible. Nous la convertissons donc en matrice dense avec la méthode `todense()`. Le résultat est donné en figure 8.

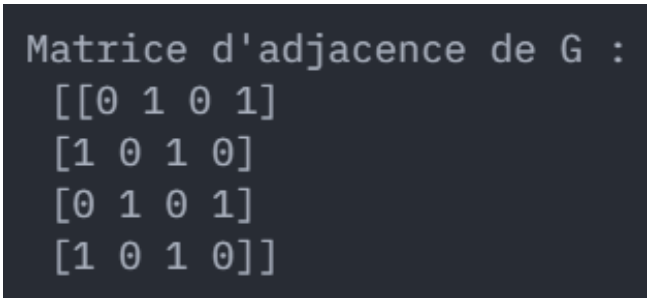


Figure 8: Matrice d'adjacence du graphe G

Les lignes et colonnes suivent l'ordre des sommets de G : 2, 3, 4, 5 (le sommet 1 ayant été supprimé). La matrice est symétrique et de diagonale nulle, car G est non orienté et sans boucle. Chaque ligne a pour somme 2, ce qui correspond au fait que chaque sommet possède exactement deux voisins.

12 Pondération des arcs du graphe DG

Nous associons à chaque arc de DG un poids entier tiré aléatoirement entre 1 et 4, à l'aide de `np.random.randint(1, 5)` (la borne supérieure étant exclue). Le poids est stocké dans l'attribut `weight` de l'arc, qui est celui lu par NetworkX lors du calcul de la matrice d'adjacence. Le résultat est donné en figure 9.

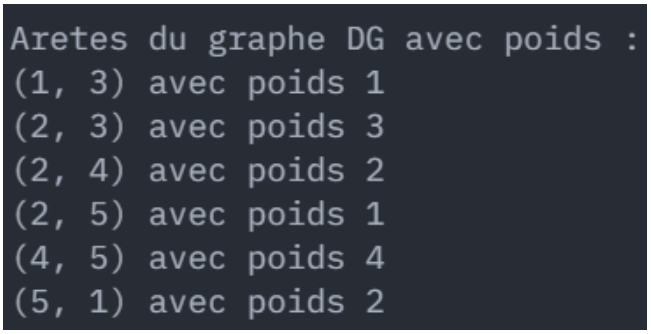


Figure 9: Pondération des arcs du graphe DG

On constate que chacun des six arcs porte un poids compris entre 1 et 4. Ces poids étant aléatoires, ils varient d'une exécution à l'autre : les matrices et listes d'adjacence présentées dans la suite correspondent aux valeurs de la figure 9.

13 Matrice d'adjacence du graphe DG

Nous calculons la matrice d'adjacence de DG avec la même fonction `adjacency_matrix()`, suivie de la même conversion en matrice dense. Comme les arcs sont pondérés, chaque case non nulle contient le poids de l'arc correspondant (figure 9) et non plus la valeur 1. Le résultat est donné en figure 10.

```
Matrice d'adjacence de DG :
 [[0 0 1 0 0]
 [0 0 3 2 1]
 [0 0 0 0 0]
 [0 0 0 0 4]
 [2 0 0 0 0]]
```

Figure 10: Matrice d'adjacence du graphe orienté DG

Les lignes et colonnes suivent l'ordre des sommets 1, 2, 3, 4, 5. Contrairement à celle de G, cette matrice n'est pas symétrique, car DG est orienté : l'arc $(5, 1)$ donne une valeur non nulle en ligne 5, colonne 1, alors que la case ligne 1, colonne 5 vaut 0. Elle contient exactement six valeurs non nulles, une par arc.

14 Listes d'adjacence du graphe DG

Nous construisons les listes d'adjacence de DG sous la forme d'un dictionnaire Python : à chaque sommet, nous associons la liste de ses successeurs, obtenue avec la méthode `neighbors()`, qui renvoie les successeurs dans un graphe orienté. Une boucle affiche ensuite une ligne par sommet. Cette représentation ne restitue que la structure du graphe, sans les poids des arcs, qui figurent dans la matrice d'adjacence de la section précédente. Le résultat est donné en figure 11.

```
Listes d'adjacence de DG :
  1: [3]
  2: [3, 4, 5]
  3: []
  4: [5]
  5: [1]
```

Figure 11: Listes d'adjacence du graphe orienté DG

On constate que le sommet 2 possède trois successeurs (3, 4 et 5), que les sommets 1, 4 et 5 en ont un chacun (respectivement 3, 5 et 1), et que le sommet 3 n'en a aucun, ce qui se traduit par une liste vide. Le total de six successeurs correspond bien aux six arcs de DG. Par rapport à la matrice, cette représentation est plus compacte, puisqu'elle ne mentionne que les arcs existants.

Part II
Parcours en profondeur

Le parcours en profondeur (*Depth-First Search*, DFS) est un algorithme de parcours de graphe qui, à partir d'un sommet de départ, explore le graphe en s'enfonçant le plus loin possible le long d'un chemin avant de revenir en arrière. Les sommets déjà visités sont marqués afin de ne pas être visités une seconde fois, ce qui évite de boucler indéfiniment. Le parcours s'arrête lorsqu'aucun sommet non visité n'est atteignable.

Dans cette partie, nous travaillons sur un graphe non orienté G à huit sommets (a à h), donné par son dessin et sa matrice d'adjacence dans l'énoncé. Nous commençons par représenter ce graphe en Python à l'aide d'un dictionnaire (question 1), puis nous écrivons la fonction de parcours en profondeur (question 2) et l'appliquons à partir des sommets a puis f (question 3).

15 Question 1 : Implémentation de G en utilisant un dictionnaire

Nous avons implémenté le graphe G de l'énoncé sous forme de **liste d'adjacence** à l'aide d'un dictionnaire Python : chaque clé est un sommet et sa valeur est la liste de ses voisins. Ce dictionnaire a été construit en lisant la matrice d'adjacence ligne par ligne (les colonnes contenant un 1 indiquent les voisins). Par exemple, la ligne du sommet e donne ses voisins b, d, f et g. Le code source est fourni dans le fichier joint.

La figure 12 montre le résultat de l'exécution, c'est-à-dire l'affichage du dictionnaire représentant G.

```
Graphe G (dictionnaire) :
a -> ['b', 'c']
b -> ['a', 'd', 'e']
c -> ['a', 'd']
d -> ['b', 'c', 'e']
e -> ['b', 'd', 'f', 'g']
f -> ['e', 'g']
g -> ['e', 'f', 'h']
h -> ['g']
```

Figure 12: Affichage du dictionnaire représentant le graphe G

Pour vérifier visuellement l'implémentation, nous avons également tracé le graphe à l'aide des bibliothèques `NetworkX` et `matplotlib` (figure 13). Il correspond bien au graphe de l'énoncé.

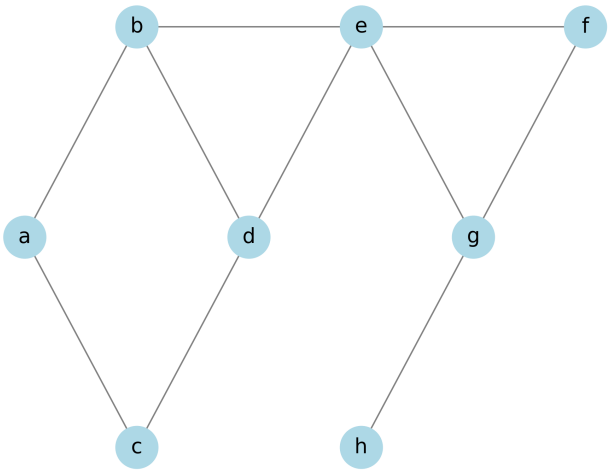


Figure 13: Tracé du graphe G construit à partir du dictionnaire

Note : dans le code, le dictionnaire `pos` associe à chaque sommet ses coordonnées (x, y) dans la figure (x : position horizontale, y : position verticale). Il est passé en paramètre à `nx.draw` afin que la disposition des sommets reproduise le dessin de l'énoncé. Sans lui, NetworkX place les sommets automatiquement, et la disposition peut changer à chaque exécution.

16 Question 2 : Fonction `ParcourirProfondeur(Graphe, Sommet, Visitee)`

Nous avons écrit une fonction `ParcourirProfondeur` qui réalise le parcours en profondeur du graphe G à partir d'un sommet donné. Elle prend trois paramètres :

- `Graphe` : le dictionnaire d'adjacence construit à la question 1
- `Sommet` : le sommet à partir duquel le parcours est lancé
- `Visitee` : la liste des sommets déjà visités, qui joue le rôle de « marque » demandé dans l'algorithme de l'énoncé.

16.1 Code de la fonction

```
def ParcourirProfondeur(Graphe, Sommet, Visitee):
    # 1. On marque le sommet courant comme "visite"
    Visitee.append(Sommet)

    # 2. On parcourt ses voisins, un par un
    for voisin in Graphe[Sommet]:
        # 3. Si le voisin n'est pas visite, on relance le parcours depuis lui
        if voisin not in Visitee:
            ParcourirProfondeur(Graphe, voisin, Visitee)

    # 4. Tous les voisins ont ete traites : on renvoie la liste
    return Visitee
```

16.2 Fonctionnement

La fonction est **récursive** : elle s'appelle elle-même sur chaque nouveau sommet rencontré, ce qui reproduit directement l'algorithme de l'énoncé. Elle se déroule en quatre étapes.

1. **Marquage du sommet de départ.** Le sommet courant est ajouté à `Visitee`, ce qui correspond à l'étape « initialement, on marque le sommet de départ comme visité ».
2. **Choix d'une arête sortante.** La boucle `for` parcourt la liste d'adjacence `Graphe[Sommet]`, c'est-à-dire les voisins du sommet courant, dans l'ordre où ils sont écrits dans le dictionnaire.
3. **Test de visite.** Pour chaque voisin, on teste `voisin not in Visitee`. S'il a déjà été visité, il est ignoré : c'est ce test qui empêche de boucler indéfiniment, puisque le graphe contient des cycles (par exemple $a \to b \to d \to c \to a$).
4. **Appel récursif.** Si le voisin n'est pas visité, la fonction est relancée à partir de lui : il sera marqué, puis ses propres voisins seront explorés. C'est ce qui fait « s'enfoncer le plus loin possible » avant de revenir en arrière.

Lorsque tous les voisins d'un sommet ont été examinés, la fonction se termine et le parcours **revient en arrière** (*backtracking*) vers le sommet qui l'avait appelée, qui poursuit avec son voisin suivant. Le parcours s'arrête lorsque le premier appel a examiné tous ses voisins : plus aucun sommet non visité n'est atteignable. La fonction renvoie alors la liste `Visitee`, qui contient les sommets **dans l'ordre de visite**.

17 Question 3 : Application de la fonction sur le graphe G

Nous appliquons maintenant la fonction `ParcourirProfondeur` sur le graphe G, d'abord en partant du sommet `a`, puis en partant du sommet `f`. Pour chaque appel, on passe une nouvelle liste vide `[]` en paramètre `Visitee`, afin que les deux parcours soient totalement indépendants : sans cela, le deuxième parcours contiendrait déjà les sommets visités lors du premier.

17.1 Code utilisé

```python
#Parcours en profondeur depuis le sommet a
print('depuis a:')
visitee_a = ParcourirProfondeur(G, 'a', [])
print(visitee_a)

#parcours en profondeur depuis le sommet f
visitee_f = ParcourirProfondeur(G, 'f', [])
print('depuis f :')
print(visitee_f)
```

Figure 14: Code utilisé pour lancer le parcours en profondeur depuis a puis depuis f

17.2 Résultat de l'exécution

```
depuis a:
['a', 'b', 'd', 'c', 'e', 'f', 'g', 'h']
depuis f :
['f', 'e', 'b', 'a', 'c', 'd', 'g', 'h']
```

Figure 15: Résultat de l'exécution du parcours en profondeur depuis a puis depuis f

17.3 Interprétation

On constate que les deux parcours visitent bien les huit sommets du graphe, mais dans un ordre différent. En partant de `a`, l'ordre obtenu est :

$$a \to b \to d \to c \to e \to f \to g \to h$$

En partant de `f`, l'ordre obtenu est :

$$f \to e \to b \to a \to c \to d \to g \to h$$

L'ordre de visite dépend donc du sommet de départ et de l'ordre dans lequel les voisins apparaissent dans le dictionnaire. Ce comportement est conforme à l'algorithme de l'énoncé : la fonction s'enfonce le plus loin possible le long d'un chemin avant de revenir en arrière lorsqu'aucun sommet non visité n'est atteignable. Le test `voisin not in Visitee` joue ici un rôle essentiel : c'est lui qui empêche le parcours de boucler indéfiniment sur les cycles du graphe, comme $a \to b \to d \to c \to a$.

Conclusion

Ce travail pratique nous a permis de passer de la théorie des graphes à leur mise en œuvre concrète en Python, en combinant l'usage d'une bibliothèque spécialisée et l'écriture personnelle d'un algorithme.

Dans la première partie, nous avons utilisé NetworkX pour créer un graphe non orienté G et un graphe orienté DG, puis gérer leurs sommets et leurs liens. Cette prise en main nous a permis de comparer arêtes et arcs, de calculer l'ordre et la taille des graphes, et d'étudier deux représentations complémentaires : la matrice d'adjacence, qui offre une lecture globale des liens, et les listes d'adjacence, plus compactes. La pondération des arcs nous a enfin montré comment enrichir un graphe en associant une donnée à chaque lien.

Dans la seconde partie, nous avons représenté le graphe par un dictionnaire et implémenté nous-mêmes une fonction récursive de parcours en profondeur. Son application depuis deux sommets différents a produit deux ordres de visite distincts, ce qui met en évidence l'influence du point de départ et de l'ordre des voisins. Le marquage des sommets déjà visités s'est révélé essentiel pour garantir la terminaison du parcours et éviter de boucler sur les cycles du graphe.

En définitive, ce TP nous a familiarisés avec les notions et les mécanismes qui sous-tendent les méthodes bio-inspirées, dont les algorithmes reposent précisément sur une représentation et une exploration efficaces des graphes.